CHAPTER 1

Language models

The preceding chapters established the computational foundations of deep learning and analyzed recurrent neural networks as dynamical systems capable of processing variable-length sequences. However, before deploying neural architectures to process or generate human text, a foundational theoretical question must be resolved: what does it mean, in formal mathematical terms, to model a natural language? Rather than treating text as arbitrary strings of alphanumeric characters or hand-crafted symbolic parse trees, modern computational linguistics conceptualizes language as a discrete stochastic process governed by an underlying, non-uniform probability distribution over sequential utterances.

The goal of language modeling is to formulate computational mechanisms capable of distinguishing between probable, semantically coherent utterances and implausible, grammatically corrupted sequences. Historically, this objective originated in the pioneering work of Claude Shannon and the engineering impasses of early speech recognition, where deterministic grammars proved fundamentally inadequate to capture the pervasive ambiguity and continuous variability of natural human expression. Early statistical approaches addressed this challenge through Markovian assumptions and discrete frequency estimation over local n-gram tuples, establishing the foundational paradigms of maximum likelihood training, likelihood evaluation, and autoregressive generation.

This chapter develops the theoretical, statistical, and algorithmic principles of language modeling. We begin by examining the historical imperatives that motivated probabilistic language modeling and formalize the joint distribution over word sequences via the probability chain rule. We then formulate the Markov assumption and n-gram models, trace parameter estimation via Maximum Likelihood Estimation on a concrete corpus, and systematically analyze the dual inference modes: sequence likelihood evaluation and autoregressive sequence generation. Finally, we dissect the fundamental limitations of statistical counting models—specifically combinatorial data sparsity, the zero-frequency collapse, and orthogonal symbolic representations—examine smoothing and discounting remedies, derive information-theoretic evaluation metrics including cross-entropy and perplexity, and establish the structural necessity of transitioning to continuous distributed word embeddings.

1.1 Introduction to language models and the modeling task

Understanding modern foundation architectures requires first examining how natural language was formalized as a statistical object. The formulation of language modeling emerged not from abstract linguistic taxonomy, but from concrete engineering challenges in telecommunications and signal processing that required machines to quantify the plausibility of textual hypotheses.

1.1.1 Historical motivation: the urge to model natural language

The mathematical formalization of language modeling originated in Claude Shannon's landmark 1948 treatise, *A Mathematical Theory of Communication* [**?**]. Shannon sought to quantify the fundamental limits of data compression and transmission across noisy telecommunication channels. Rather than viewing text as static philosophical prose, Shannon conceptualized natural language as a stochastic process generating discrete sequences of symbols chosen from a finite alphabet according to stationary statistical probabilities. By measuring the entropy of printed English through letter-level and word-level approximation experiments, Shannon demonstrated that natural text possesses substantial statistical redundancy, implying that preceding context exerts powerful probabilistic constraints on upcoming symbols.

In the decades that followed, the imperative to build automated language models transitioned from theoretical information theory to critical real-world signal processing domains:

1. **Acoustic speech recognition (ASR)**: In automated speech transcription, an input acoustic waveform must be mapped to a written sequence of words. However, human phonetics exhibits pervasive phonetic ambiguity: homophones and continuous connected speech yield acoustic signals that correspond equally well to multiple disparate word sequences. For example, the acoustic sequence corresponding to the phonetic sequence `/rek-uhg-nyze-beech/` can be transcribed as either *"recognize speech"* or *"wreck a nice beach"*. An acoustic sensor analyzing frequency spectra alone cannot reliably resolve this ambiguity. To achieve human-level accuracy, an ASR pipeline must factorize the problem into two complementary components:

$$W^* = \arg\max_W P(W \mid A) = \arg\max_W \frac{P(A \mid W)P(W)}{P(A)} = \arg\max_W P(A \mid W)P(W), \tag{1.1}$$

   where $P(A \mid W)$ denotes the acoustic likelihood model (how likely the sound A is given words W), and $P(W)$ represents the language model prior. Because the phrase *"recognize speech"* occurs frequently in technical discourse while *"wreck a nice beach"* is exceptionally rare, the prior enforces $P(\text{"recognize speech"}) \gg P(\text{"wreck a nice beach"})$, allowing the system to effortlessly resolve phonetic noise.

2. **Machine translation (MT) and optical character recognition (OCR)**: In machine translation, transferring syntactic structures between disparate languages rarely proceeds via word-for-word substitution. An automated translator generating candidate lexical mappings requires a statistical prior over the target language to select phrase orderings that sound natural and grammatically fluent. Similarly, in optical character recognition, noisy visual scans that render ambiguous characters (such as distinguishing `'l'`, `'1'`, and `'I'`) are disambiguated by evaluating which lexical candidate forms a coherent word sequence within the sentence.

These engineering requirements triggered a philosophical pivot in computational linguistics. Early artificial intelligence research, guided by Chomskyan generative grammars [**?**], attempted to model natural language through deterministic, handcrafted formal rule sets that partitioned sentences strictly into grammatically valid or invalid categories. However, human language constantly breaks rigid grammatical rules through idioms, colloquialisms, poetry, and colloquial

speech while remaining completely intelligible. Statistical language modeling replaced this binary grammaticality dichotomy with a continuous, probabilistic spectrum of plausibility.

1.1.2 The language model task: formal probability distribution

Formally, let $\mathcal{V}$ denote a finite vocabulary comprising all discrete token primitives recognized by the system, with cardinality $|\mathcal{V}|$. An arbitrary natural language utterance W is defined as an ordered tuple of T tokens:

$$W = (w_1, w_2, \ldots, w_T), \quad w_t \in \mathcal{V}, \quad T \in \mathbb{N}^+. \tag{1.2}$$

The set of all possible finite sequences formed from vocabulary $\mathcal{V}$ is given by the Kleene closure $\mathcal{V}^* = \bigcup_{k=1}^{\infty} \mathcal{V}^k$.

A *Language Model* (LM) is a mathematical function that assigns a joint probability $P(W)$ to every possible sequence $W \in \mathcal{V}^*$, satisfying the standard Kolmogorov probability axioms:

$$P(W) \geq 0 \quad \forall W \in \mathcal{V}^*, \quad \text{and} \quad \sum_{W \in \mathcal{V}^*} P(W) = 1. \tag{1.3}$$

The probability distribution $P(W)$ can be conceptualized as defining a continuous density landscape across the discrete space of possible word combinations. In this space:

- **High-probability regions**: Encompass plausible, fluent, and syntactically well-formed sentences that frequently occur in human communication:

$$P(\texttt{"This is a reasonable sentence"}) \approx 10^{-1}. \tag{1.4}$$

- **Low-probability regions**: Encompass syntactically well-formed but semantically anomalous or bizarre sequences that violate common semantic expectations:

$$P(\texttt{"This is a purple sentence"}) \approx 10^{-3}. \tag{1.5}$$

- **Near-zero probability regions**: Encompass ungrammatical, disjoint, or corrupted strings of symbols that violate the structural syntax of the language:

$$P(\texttt{"This are a reasonable sentence"}) \approx 10^{-5}. \tag{1.6}$$

Because the combinatorial space of arbitrary symbol permutations is astronomically large, the overwhelming majority of sequences in $\mathcal{V}^*$ are ungrammatical noise. Consequently, natural human language occupies an extremely narrow, low-dimensional probability manifold within $\mathcal{V}^*$. The fundamental objective of language modeling is to accurately estimate this non-uniform probability distribution from finite empirical observations.

1.1.3 Factoring joint probabilities via the chain rule

Directly estimating the joint probability $P(w_1, w_2, \ldots, w_T)$ over an entire sentence simultaneously is computationally intractable because the number of possible complete sentences grows exponentially with sequence length T. To render this estimation feasible, we apply the general *chain rule of probability* (or Kolmogorov multiplication law), which factors any joint distribution into a product of conditional distributions:

$$\begin{aligned} P(W) &= P(w_1, w_2, \ldots, w_T) \\ &= P(w_1)\, P(w_2 \mid w_1)\, P(w_3 \mid w_1, w_2) \cdots P(w_T \mid w_1, \ldots, w_{T-1}) \\ &= \prod_{t=1}^{T} P(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{T} P(w_t \mid w_{<t}), \end{aligned} \tag{1.7}$$

where $w_{<t} = (w_1, \ldots, w_{t-1})$ denotes the sequence prefix comprising all tokens preceding step t, with the base case defined as $P(w_1 \mid w_{<1}) = P(w_1)$.

Equation (1.7) provides a profound conceptual and operational shift. It demonstrates that the problem of assigning a joint probability to an entire sentence is mathematically identical to the sequential task of predicting the conditional probability distribution of the single next word w_t given the observed historical context $w_{<t}$.

This formulation directly aligns language modeling with the classical *Cloze* (fill-in-the-blank) evaluation methodology in cognitive linguistics:

Cloze prompt 1: `"This is a reasonable [ ]"` $\implies$ $P\,(w_t = \texttt{"sentence"} \mid \texttt{"This is a reasonable"})$ (1.8)

Cloze prompt 2: `"The black cat sat on the [ ]"` $\implies$ $P\,(w_t = \texttt{"mat"} \mid \texttt{"The black cat sat on th}$ (1.9)

By framing language modeling as sequential next-token prediction, we can construct models that process text incrementally from left to right, evaluating sentence plausibility and generating synthetic continuations through successive conditional probability evaluations.

1.2 The Markov assumption, n-gram models, and operational inference

While Equation (1.7) is mathematically exact, it cannot be implemented directly in practice using raw frequency tables. As sequence length t increases, the conditioning prefix $w_{<t}$ grows linearly, meaning that the historical context becomes unique. To construct practical statistical models, we must introduce simplifying independence assumptions that truncate the conditioning horizon.

1.2.1 The Markov assumption and combinatorial reduction

Consider estimating the conditional probability $P(w_t \mid w_1, \ldots, w_{t-1})$ using empirical counting over a text corpus. For a vocabulary of size $|\mathcal{V}| = 50{,}000$, a prefix of length $t - 1 = 10$ admits $|\mathcal{V}|^{10} \approx 9.76 \times 10^{46}$ distinct possible histories. Because no realistic training corpus contains more than 10^{10} to 10^{12} total words, the probability of encountering any specific 10-word prefix multiple times approaches zero, rendering empirical frequency ratios completely undefined.

To overcome this combinatorial barrier, statistical language modeling applies the *Markov assumption* [**?**]. Formally, an $(n-1)$-th order Markov chain posits that the conditional probability of the current state depends exclusively on the immediately preceding $n-1$ states, remaining conditionally independent of all earlier history:

$$P\left(w_t \mid w_1, w_2, \ldots, w_{t-1}\right) \approx P\left(w_t \mid w_{t-n+1}, \ldots, w_{t-1}\right). \tag{1.10}$$

Under the $(n-1)$-th order Markov assumption, an n-gram language model approximates the true joint probability of a sequence as:

$$P\left(w_1, \ldots, w_T\right) \approx \prod_{t=1}^{T} P\left(w_t \mid w_{t-n+1}, \ldots, w_{t-1}\right). \tag{1.11}$$

The choice of parameter n defines the context horizon of the statistical model:

1. **Unigram model ($n = 1$)**: Assumes complete zeroth-order independence. The probability of each word is evaluated in total isolation from its context:

$$P\left(w_1, \ldots, w_T\right) \approx \prod_{t=1}^{T} P\left(w_t\right). \tag{1.12}$$

   A unigram model ignores all syntactic structure and word order, treating sentences as an unordered "bag of words".

2. **Bigram model ($n = 2$)**: Imposes a first-order Markov assumption. The probability of the next word depends strictly and exclusively on the single immediately preceding word:

$$P\left(w_1, \ldots, w_T\right) \approx P\left(w_1\right) \prod_{t=2}^{T} P\left(w_t \mid w_{t-1}\right). \tag{1.13}$$

3. **Trigram model ($n = 3$)**: Imposes a second-order Markov assumption, conditioning each word on the preceding two-word prefix:

$$P\left(w_1, \ldots, w_T\right) \approx P\left(w_1\right) P\left(w_2 \mid w_1\right) \prod_{t=3}^{T} P\left(w_t \mid w_{t-2}, w_{t-1}\right). \tag{1.14}$$

1.2.2 The worked transition matrix and parameter estimation

By truncating the conditioning horizon to a small constant n, the parameters of an n-gram model can be estimated directly from a reference corpus using *Maximum Likelihood Estimation* (MLE). Under MLE, the parameter values are chosen to maximize the likelihood of the observed training text, which equates to computing empirical frequency ratios.

For a bigram model ($n = 2$), the conditional probability of token w_t given predecessor w_{t-1} is evaluated by counting the joint occurrences of the bigram (w_{t-1}, w_t) and dividing by the unigram frequency of the prefix w_{t-1}:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{\sum_{w \in \mathcal{V}} C(w_{t-1}, w)} = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})}, \tag{1.15}$$

where $C(\cdot)$ denotes the count of exact occurrences in the training corpus.

Worked pedagogical example. To ground this formulation in concrete numbers, we analyze the exact toy corpus evaluated during course lectures. Consider a tiny reference corpus C consisting of two sentences:

$$\text{Sentence 1:} \quad \texttt{"The cat chased the mouse happily."} \tag{1.16}$$

$$\text{Sentence 2:} \quad \texttt{"The mouse ate the cheese."} \tag{1.17}$$

Extracting all unique lexical tokens yields a vocabulary of cardinality $|\mathcal{V}| = 7$:

$$\mathcal{V} = \{\texttt{ate}, \texttt{cat}, \texttt{chased}, \texttt{cheese}, \texttt{happily}, \texttt{mouse}, \texttt{the}\}. \tag{1.18}$$

We construct the empirical transition matrix by scanning all adjacent bigram pairs across the corpus. Scanning Sentence 1 yields the bigram sequence:

$$(\texttt{the}, \texttt{cat}),\ (\texttt{cat}, \texttt{chased}),\ (\texttt{chased}, \texttt{the}),\ (\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{happily}). \tag{1.19}$$

Scanning Sentence 2 yields:

$$(\texttt{the}, \texttt{mouse}),\ (\texttt{mouse}, \texttt{ate}),\ (\texttt{ate}, \texttt{the}),\ (\texttt{the}, \texttt{cheese}). \tag{1.20}$$

Summing these occurrences yields the absolute frequency matrix $C(w_{t-1}, w_t)$. Normalizing each row by the unigram count of the conditioning prefix ensures that $\sum_{w_t \in \mathcal{V}} P(w_t \mid w_{t-1}) = 1.0$ for all non-terminal rows, resulting in the complete bigram transition matrix presented in Table 1.1.

$w_{t-1} \setminus w_t$	**ate**	**cat**	**chased**	**cheese**	**happily**	**mouse**	**the**
ate	0	0	0	0	0	0	1.00 (1/1)
cat	0	0	1.00 (1/1)	0	0	0	0
chased	0	0	0	0	0	0	1.00 (1/1)
cheese	0	0	0	0	0	0	0
happily	0	0	0	0	0	0	0
mouse	0.50 (1/2)	0	0	0	0.50 (1/2)	0	0
the	0	0.25 (1/4)	0	0.25 (1/4)	0	0.50 (2/4)	0

Table 1.1: Empirical Maximum Likelihood Estimation (MLE) bigram transition probability matrix computed from the lecture reference corpus. Each entry (i, j) represents the conditional probability $P(w_j \mid w_i) = C(w_i, w_j)/C(w_i)$. Rows for terminal words (`cheese`, `happily`) sum to zero in the absence of explicit sentence boundary delimiters.

Inspecting Table 1.1 illustrates the statistical profile learned by the model:

- Following the word `"the"`, the model distributes probability mass across three nouns: $P(\texttt{mouse} \mid \texttt{the}) = 0.50$, $P(\texttt{cat} \mid \texttt{the}) = 0.25$, and $P(\texttt{cheese} \mid \texttt{the}) = 0.25$.
- Following the word `"mouse"`, the model assigns equal probability to the verb `"ate"` and the adverb `"happily"`: $P(\texttt{ate} \mid \texttt{mouse}) = 0.50$, $P(\texttt{happily} \mid \texttt{mouse}) = 0.50$.
- Deterministic transitions emerge where a word was observed with only a single successor: $P(\texttt{chased} \mid \texttt{cat}) = 1.00$, $P(\texttt{the} \mid \texttt{chased}) = 1.00$, and $P(\texttt{the} \mid \texttt{ate}) = 1.00$.

Once the transition matrix is populated and normalized, it functions as a complete, trained statistical language model. We can deploy this model in two fundamental operational modes: sequence likelihood evaluation and autoregressive text generation.

1.2.3 Inference: evaluating sentence likelihood

The first core inference task consists in evaluating the likelihood of a candidate word sequence. Given an unobserved test sentence, the language model computes its joint probability by taking the product of the corresponding transition probabilities from Table 1.1.

Worked numerical evaluation. Suppose we deploy the model to evaluate the probability of a candidate sentence that did not appear in the training corpus:

$$W_{\text{test}} = \texttt{"The cat chased the cheese."} \tag{1.21}$$

Applying the bigram factorization from Equation (1.11):

$$P\left(W_{\text{test}}\right) = P\left(\texttt{the}\right) \cdot P\left(\texttt{cat} \mid \texttt{the}\right) \cdot P\left(\texttt{chased} \mid \texttt{cat}\right) \cdot P\left(\texttt{the} \mid \texttt{chased}\right) \cdot P\left(\texttt{cheese} \mid \texttt{the}\right). \tag{1.22}$$

Assuming the unigram prior for the sentence-initial token is $P(\texttt{the}) = 1.0$ (since all training sentences in corpus C begin with `"The"`), we substitute the empirical conditional probabilities from Table 1.1:

$$\begin{aligned} P\left(W_{\text{test}}\right) &= 1.0 \times \frac{1}{4} \times 1.0 \times 1.0 \times \frac{1}{4} \\ &= 1.0 \times 0.25 \times 1.0 \times 1.0 \times 0.25 = \frac{1}{16} = 0.0625. \end{aligned} \tag{1.23}$$

The model assigns a non-zero probability ($P = 0.0625$) to *"The cat chased the cheese"*. Crucially, this demonstrates that statistical language models can assign valid probability mass to novel combinations of words that were never observed verbatim during training, provided that all constituent transitions exist in the learned transition graph.

Numerical stability via log-likelihood. In practical applications involving sentences of realistic length ($T \geq 20$), multiplying dozens of conditional probabilities bounded within $[0, 1]$ leads directly to numerical underflow:

$$P(W) = \prod_{t=1}^{T} P\left(w_t \mid w_{t-1}\right) < 10^{-308}, \tag{1.24}$$

exceeding the precision of standard 64-bit IEEE double-precision floating-point arithmetic. To guarantee numerical stability, all probability computations are evaluated in the logarithmic domain:

$$\log P(W) = \log P\left(w_1\right) + \sum_{t=2}^{T} \log P\left(w_t \mid w_{t-1}\right). \tag{1.25}$$

Operating in log-space transforms unstable products of small fractions into stable sums of negative real numbers, preserving full analytical fidelity across sequences of arbitrary length.

1.2.4 Inference: autoregressive text generation and decoding dynamics

The second operational mode of a language model is *autoregressive text generation*. In this regime, the model does not score an existing sentence; instead, it synthesizes new text token by token. At each step t, the model conditions on the antecedent token w_{t-1}, retrieves the conditional categorical distribution $P(\cdot \mid w_{t-1})$ from its transition matrix, samples a token w_t, appends w_t to the generated sequence, and feeds w_t back into the model to condition the subsequent step:

$$w_t \sim P\left(\cdot \mid w_{t-1}\right). \tag{1.26}$$

Step-by-step lecture generation trace. We trace the complete generative execution evaluated in lecture, initiated with the prompt token w_1 = `"The"`:

1. **Step 1**: Prompt w_1 = `"The"`. The model queries row `"the"` in Table 1.1, obtaining the categorical distribution:

$$P\left(\texttt{cat} \mid \texttt{the}\right) = 0.25, \quad P\left(\texttt{cheese} \mid \texttt{the}\right) = 0.25, \quad P\left(\texttt{mouse} \mid \texttt{the}\right) = 0.50. \tag{1.27}$$

   Sampling stochastically from this distribution selects the candidate w_2 = `"mouse"`.

$$\text{Current sequence:} \quad \texttt{"The mouse"} \tag{1.28}$$

2. **Step 2**: Preceding token w_2 = `"mouse"`. The model queries row `"mouse"`:

$$P\left(\texttt{ate} \mid \texttt{mouse}\right) = 0.50, \quad P\left(\texttt{happily} \mid \texttt{mouse}\right) = 0.50. \tag{1.29}$$

   Sampling from this distribution selects w_3 = `"ate"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate"} \tag{1.30}$$

3. **Step 3**: Preceding token w_3 = `"ate"`. The model queries row `"ate"`:

$$P\left(\texttt{the} \mid \texttt{ate}\right) = 1.00. \tag{1.31}$$

   The transition is fully deterministic, yielding w_4 = `"the"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the"} \tag{1.32}$$

4. **Step 4**: Preceding token w_4 = `"the"`. The model again queries row `"the"`. Although the model previously generated `"mouse"` at step 2, the bigram Markov assumption possesses zero memory of steps 1 to 3. Sampling from the identical distribution selects w_5 = `"mouse"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse"} \tag{1.33}$$

5. **Step 5**: Preceding token w_5 = `"mouse"`. Consulting row `"mouse"`, the model samples w_6 = `"happily"`.

$$\text{Current sequence:} \quad \texttt{"The mouse ate the mouse happily"} \tag{1.34}$$

6. **Step 6**: Preceding token w_6 = `"happily"`. Row `"happily"` contains strictly zero counts across all columns ($\sum_w C(\texttt{happily}, w) = 0$). In the transition graph, `"happily"` represents an *absorbing terminal state*. The model has no valid successor tokens to sample, and generation halts.

The final generated utterance is:

$$\texttt{"The mouse ate the mouse happily."} \tag{1.35}$$

This sentence never existed in the training corpus; it was synthesized dynamically through iterative sampling along the stochastic transition graph.

Decoding strategies and pathological dynamics. The generation trace highlights two critical operational aspects of autoregressive generation:

1. **Absorbing terminal states and explicit boundary modeling**: In the toy example, generation halted because the terminal word `"happily"` had zero outgoing transitions. In rigorous natural language processing, stopping conditions are formalized by augmenting the vocabulary with dedicated boundary tokens: `<BOS>` (Beginning of Sentence) and `<EOS>` (End of Sentence). Training sentences are padded as `<BOS>` $w_1 \ \ldots \ w_T$ `<EOS>`. The model learns explicit transition probabilities $P(\texttt{<EOS>} \mid w_t)$, allowing the network to decide autonomously when an utterance is complete.

2. **Greedy decoding versus stochastic sampling and cyclic limit cycles**: Instead of sampling stochastically, an alternative decoding strategy is *greedy decoding*, where the model deterministically selects the single token with maximum conditional probability:

$$w_t^* = \arg\max_{w \in \mathcal{V}} P\left(w \mid w_{t-1}\right). \tag{1.36}$$

   However, when greedy decoding is applied to finite transition matrices, the model frequently falls into *cyclic limit cycles*. In Table 1.1, starting from `"the"`, the greedy choice is w_2 = `"mouse"` ($P = 0.50$). If ties are broken in favor of `"ate"`, the sequence proceeds `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"` $\rightarrow$ `"the"` $\rightarrow$ `"mouse"` $\rightarrow$ `"ate"` $\ldots$, trapping the model in an infinite periodic loop. This pathology persists in modern foundation models, necessitating advanced decoding heuristics such as temperature scaling, top-k sampling, and nucleus (top-p) truncation.

1.3 Limitations of statistical language models and the path to neural representations

Despite their mathematical simplicity and clear probabilistic interpretations, statistical n-gram models suffer from severe theoretical, combinatorial, and representational pathologies that fundamentally limit their scalability.

1.3.1 Combinatorial data sparsity and the zero-frequency collapse

The most fatal pathology of n-gram models is the curse of combinatorial data sparsity. An n-gram model over a realistic vocabulary $\mathcal{V}$ requires maintaining a contingency table whose parameter count scales exponentially with the context window size:

$$N_{\text{parameters}} = |\mathcal{V}|^n. \tag{1.37}$$

For a standard vocabulary of size $|\mathcal{V}| = 50{,}000$:

- **Bigram ($n = 2$)**: Requires $|\mathcal{V}|^2 = 2.5 \times 10^9$ parameters (requiring ≈ 10 GB of memory under 32-bit floats).
- **Trigram ($n = 3$)**: Requires $|\mathcal{V}|^3 = 1.25 \times 10^{14}$ parameters (requiring ≈ 500 TB of storage).
- **5-gram ($n = 5$)**: Requires $|\mathcal{V}|^5 = 3.125 \times 10^{23}$ parameters, far exceeding the physical storage capacity of global computing infrastructure.

Because human language allows words to combine in virtually boundless configurations, no finite corpus can ever observe more than a microscopic fraction of all valid n-grams. In any empirical training set, the vast majority of legitimate word combinations have an observed count of zero:

$$C(w_{t-n+1}, \ldots, w_t) = 0. \tag{1.38}$$

Under Maximum Likelihood Estimation, if a specific bigram (w_{t-1}, w_t) was never seen during training, its assigned probability is identically zero:

$$P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{0}{C(w_{t-1})} = 0. \tag{1.39}$$

This leads directly to the *zero-frequency collapse*: when evaluating a candidate test sentence $W = (w_1, \ldots, w_T)$, if even a single constituent bigram is unobserved in the training corpus, the entire multiplicative chain collapses to zero:

$$P(W) = \prod_{t=1}^{T} P(w_t \mid w_{t-1}) = 0. \tag{1.40}$$

In log-space, $\log P(W) = \log 0 = -\infty$. Consequently, a model that encounters a single unobserved word combination declares the entire sequence impossible, causing evaluation metrics such as cross-entropy and perplexity to explode to infinity.

1.3.2 Smoothing and discounting techniques

To prevent probability collapse, statistical NLP developed *smoothing* (or discounting) algorithms. Smoothing techniques shave off a small fraction of probability mass from frequently observed events and reallocate it to unobserved events, ensuring that $P(w_t \mid w_{t-1}) > 0$ for all possible token combinations.

Laplace and additive smoothing

The simplest smoothing formulation is *Laplace (Add-1) smoothing*, which artificially increments the count of every possible n-gram by 1:

$$P_{\text{Laplace}}(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + 1}{\sum_{w \in \mathcal{V}} (C(w_{t-1}, w) + 1)} = \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + |\mathcal{V}|}. \tag{1.41}$$

A generalized variant, *additive (α) smoothing* (or Lidstone smoothing), adds a fractional pseudo-count $0 < \alpha < 1$:

$$P_\alpha(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + \alpha}{C(w_{t-1}) + \alpha|\mathcal{V}|}. \tag{1.42}$$

Analytical critique of Laplace smoothing. While Laplace smoothing guarantees that no probability evaluates to zero, it distorts probability distributions disastrously when vocabulary size $|\mathcal{V}|$ is large. Suppose $|\mathcal{V}| = 50{,}000$ and an observed word appears with unigram count $C(w_{t-1}) = 100$. The denominator in Equation (1.41) increases from 100 to $100+50{,}000 = 50{,}100$. Consequently, an observed bigram that appeared 5 times has its estimated probability reduced from $5/100 = 0.05$ to $6/50{,}100 \approx 0.00012$, attenuating genuine empirical evidence by over 99%. Laplace smoothing steals far too much probability mass from observed events and scatters it indiscriminately across billions of unobserved combinations.

Good-Turing discounting

To establish a more principled redistribution of probability mass, the *Good-Turing estimator* [**?**] utilizes the frequency of other frequencies. Let N_r denote the number of distinct n-grams that were observed exactly r times in the training corpus. Good-Turing re-estimates the effective count r^* for an n-gram observed r times as:

$$r^* = (r+1)\frac{N_{r+1}}{N_r}. \tag{1.43}$$

The total probability mass reserved for all unseen events ($r = 0$) is governed by the proportion of singleton n-grams (events seen exactly once, N_1):

$$P_{\text{GT}}(\text{unseen}) = \frac{N_1}{N}, \tag{1.44}$$

where $N = \sum_{r=1}^{\infty} rN_r$ is the total token count of the corpus. Good-Turing grounds probability estimation in empirical discovery rates: the likelihood of encountering an unobserved token is proportional to the number of tokens observed only once.

Backoff and linear interpolation

When higher-order n-gram counts are scarce, models can borrow statistical strength from lower-order distributions. Two primary strategies exist:

1. **Jelinek-Mercer linear interpolation**: Computes a convex linear combination of trigram, bigram, and unigram probabilities:

$$P_{\text{interp}}(w_t \mid w_{t-2}, w_{t-1}) = \lambda_3 P_{\text{MLE}}(w_t \mid w_{t-2}, w_{t-1}) + \lambda_2 P_{\text{MLE}}(w_t \mid w_{t-1}) + \lambda_1 P_{\text{MLE}}(w_t), \tag{1.45}$$

   where interpolation coefficients satisfy $\sum_{i=1}^{3} \lambda_i = 1$ and $\lambda_i > 0$.

2. **Katz backoff**: Prioritizes the highest-order model whenever empirical counts exceed zero, backing off to discounted lower-order estimates only when the higher-order count is zero [**?**]:

$$P_{\text{Katz}}\left(w_t \mid w_{t-n+1}^{t-1}\right) = \begin{cases} P^*\left(w_t \mid w_{t-n+1}^{t-1}\right) & \text{if } C\left(w_{t-n+1}^{t}\right) > 0, \\ \alpha\left(w_{t-n+1}^{t-1}\right) P_{\text{Katz}}\left(w_t \mid w_{t-n+2}^{t-1}\right) & \text{if } C\left(w_{t-n+1}^{t}\right) = 0, \end{cases} \tag{1.46}$$

where P^* denotes discounted probabilities and $\alpha(\cdot)$ is a normalization factor that ensures probabilities sum to 1.

Kneser-Ney smoothing and continuation probabilities

The gold standard statistical smoothing algorithm is *Modified Kneser-Ney smoothing* [**?**]. Kneser-Ney recognizes a critical flaw in standard backoff: lower-order unigram frequency $P(w)$ does not reflect the likelihood of a word completing an unfamiliar context.

Consider the proper noun `"Francisco"`. In a general news corpus, `"Francisco"` has a relatively high unigram count C(`"Francisco"`) because it occurs frequently in the bigram `"San Francisco"`. Under standard Jelinek-Mercer interpolation, if the model evaluates an unfamiliar prefix such as *"I want to eat a [ ]"*, the high unigram count of `"Francisco"` would cause the model to assign it substantial probability. However, in human language, `"Francisco"` almost never appears unless preceded by `"San"`.

Kneser-Ney resolves this by replacing raw unigram frequency with *continuation probability*, $P_{\text{cont}}(w)$, which measures how versatile a word is across diverse antecedents:

$$P_{\text{cont}}(w) = \frac{|\{w' : C(w', w) > 0\}|}{\sum_{w'' \in \mathcal{V}} |\{w' : C(w', w'') > 0\}|}. \tag{1.47}$$

Because `"Francisco"` appears almost exclusively after the single predecessor `"San"`, its continuation count is 1, resulting in a tiny continuation probability P_{cont}(`"Francisco"`). Conversely, words like `"apple"` or `"book"` follow hundreds of distinct words, receiving high continuation probabilities and correctly dominating lower-order backoff distributions.

1.3.3 Intrinsic evaluation metrics: cross-entropy and perplexity

To benchmark and compare different language models quantitatively without executing costly downstream applications, natural language processing relies on intrinsic, information-theoretic metrics: *Cross-Entropy* and *Perplexity* [**? ?**].

Information-theoretic foundations: entropy and KL divergence

Let $\mathcal{X}$ denote a discrete sample space (the vocabulary $\mathcal{V}$), and let $P(x)$ represent the true, underlying, but inaccessible probability distribution of human language.

Definition 1.1: *Shannon entropy*

The *Shannon entropy* $H(P)$ measures the fundamental, irreducible uncertainty or average information content inherent in distribution P [**?**]:

$$H(P) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) = \mathbb{E}_{x \sim P}\left[\log_2 \frac{1}{P(x)}\right]. \tag{1.48}$$

By Shannon's source coding theorem, $H(P)$ represents the absolute theoretical lower bound on the expected number of bits required to encode a symbol emitted by P without

loss.

When constructing a language model, we approximate the true distribution P with an estimated parameterized distribution Q_θ. To quantify the statistical discrepancy between the true distribution P and our approximating model Q_θ, we employ the *Kullback-Leibler (KL) divergence*.

Definition 1.2: *Kullback-Leibler (KL) divergence*

The *KL divergence* (or relative entropy) from model distribution Q to target distribution P is:

$$D_{\mathrm{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{P(x)}{Q(x)} \right) = \mathbb{E}_{x \sim P} \left[\log_2 \frac{P(x)}{Q(x)} \right]. \tag{1.49}$$

Theorem 1.1: *Gibbs' inequality*

For any two discrete probability distributions P and Q, the KL divergence is non-negative:

$$D_{\mathrm{KL}}(P \parallel Q) \geq 0, \tag{1.50}$$

with equality $D_{\mathrm{KL}}(P \parallel Q) = 0$ if and only if $P(x) = Q(x)$ for all $x \in \mathcal{X}$.

Proof. Expressing the divergence via the natural logarithm and utilizing the standard inequality $\ln(u) \leq u - 1$ for all $u > 0$ (with equality if and only if $u = 1$):

$$\begin{aligned} -D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 \left(\frac{Q(x)}{P(x)} \right) = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \left(\frac{Q(x)}{P(x)} \right) \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) \\ &= \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = \frac{1}{\ln 2} (1 - 1) = 0. \end{aligned} \tag{1.51}$$

Multiplying both sides by -1 reverses the inequality, yielding $D_{\mathrm{KL}}(P \parallel Q) \geq 0$.

Cross-entropy decomposition and empirical sequence loss

Expanding the logarithm in the definition of KL divergence demonstrates the fundamental relationship connecting Entropy, KL Divergence, and Cross-Entropy:

$$\begin{aligned} D_{\mathrm{KL}}(P \parallel Q) &= \sum_{x \in \mathcal{X}} P(x) \log_2 P(x) - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \\ &= -H(P) + \left(- \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) \right). \end{aligned} \tag{1.52}$$

Rearranging terms defines the *Cross-Entropy* $H(P, Q)$:

$$H(P, Q) \triangleq - \sum_{x \in \mathcal{X}} P(x) \log_2 Q(x) = H(P) + D_{\mathrm{KL}}(P \parallel Q). \tag{1.53}$$

Equation (1.53) provides the theoretical justification for cross-entropy minimization during training. Because the true language distribution P is determined by human linguistic behavior, its entropy $H(P)$ is an invariant physical constant. When minimizing cross-entropy with respect to model parameters θ:

$$\arg\min_{\theta} H(P, Q_\theta) = \arg\min_{\theta} (H(P) + D_{\mathrm{KL}}(P \parallel Q_\theta)) = \arg\min_{\theta} D_{\mathrm{KL}}(P \parallel Q_\theta). \tag{1.54}$$

Minimizing cross-entropy is mathematically identical to driving the KL divergence between model Q_θ and the true language distribution P to zero.

Because we do not have closed-form access to the true distribution $P(x)$, the expected value is estimated empirically by sampling a long test sequence $W = (w_1, w_2, \ldots, w_T) \sim P$:

$$\mathcal{H}(W) = -\frac{1}{T} \log_2 Q(w_1, w_2, \ldots, w_T) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 Q(w_t \mid w_{<t}). \tag{1.55}$$

Empirical cross-entropy measures the average number of bits required to compress each token of test text using model Q.

Perplexity as an intrinsic evaluation metric

While cross-entropy evaluates performance on a logarithmic scale, researchers and practitioners report *Perplexity* (PPL), defined as the exponentiated cross-entropy loss:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{-\frac{1}{T}\sum_{t=1}^{T} \log_2 Q(w_t|w_{<t})} \tag{1.56}$$

$$= \left(\prod_{t=1}^{T} \frac{1}{Q(w_t \mid w_{<t})} \right)^{\frac{1}{T}}. \tag{1.57}$$

Perplexity represents the reciprocal geometric mean of the token probabilities assigned by the model across the test corpus.

Theorem 1.2: *Perplexity of a uniform K-ary distribution*

Let a language model Q predict next tokens such that at each timestep t, the probability mass is distributed uniformly across exactly K equally likely candidate tokens ($Q(w_t \mid w_{<t}) = \frac{1}{K}$ for each candidate). Then the perplexity on any generated sequence W of length T is identically K:

$$\mathrm{PPL}(W) = K. \tag{1.58}$$

Proof. Substituting the uniform prediction probability $Q(w_t \mid w_{<t}) = \frac{1}{K}$ into the empirical cross-entropy formula yields:

$$\mathcal{H}(W) = -\frac{1}{T} \sum_{t=1}^{T} \log_2 \left(\frac{1}{K} \right) = -\frac{1}{T} \sum_{t=1}^{T} (-\log_2 K)$$

$$= \frac{1}{T} (T \log_2 K) = \log_2 K. \tag{1.59}$$

Exponentiating the cross-entropy:

$$\mathrm{PPL}(W) = 2^{\mathcal{H}(W)} = 2^{\log_2 K} = K. \tag{1.60}$$

Theorem 1.2 establishes the physical interpretation of perplexity: it quantifies the *effective branching factor* of a language model. When a model evaluates a text corpus and achieves PPL = 42, this indicates that, on average, the model's predictive uncertainty at each token is mathematically equivalent to choosing uniformly at random from a shortlist of 42 equally plausible words. A uniform random guesser across vocabulary $\mathcal{V}$ achieves PPL = $|\mathcal{V}|$ (for example, PPL = 50,000), while an optimal oracle that predicts every token with probability 1.0 achieves the theoretical lower bound PPL = 1.0.

Logarithmic base conversions: bits versus nats

A practical mathematical distinction arises when connecting information-theoretic formulations (which employ base-2 logarithms) with deep learning frameworks such as PyTorch (which compute natural base-e cross-entropy loss):

Property	Information Theory (Bits)	Deep Learning / PyTorch (Nats)
Loss Formula	$\mathcal{H}_2 = -\frac{1}{T}\sum \log_2 Q(w_t \mid w_{<t})$	$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum \ln Q(w_t \mid w_{<t})$
Unit	Bits per token	Nats per token
Perplexity Formula	$\text{PPL} = 2^{\mathcal{H}_2}$	$\text{PPL} = \exp(\mathcal{L}_{\text{CE}}) = e^{\mathcal{L}_{\text{CE}}}$
Conversion Identity	$\mathcal{H}_2 = \frac{\mathcal{L}_{\text{CE}}}{\ln 2} \approx \frac{\mathcal{L}_{\text{CE}}}{0.693147}$,	$\mathcal{L}_{\text{CE}} = \mathcal{H}_2 \cdot \ln 2 \approx 0.693147 \cdot \mathcal{H}_2$

To observe the effect of this conversion, suppose a neural model trained on code completion reports an evaluation loss of $\mathcal{L}_{\text{CE}} = 2.7726$ nats. Computing base-e perplexity directly yields:

$$\text{PPL} = e^{2.7726} = \exp(2.7726) \approx 16.00. \tag{1.61}$$

Converting the loss to information bits produces:

$$\mathcal{H}_2 = \frac{2.7726}{\ln 2} = \frac{2.7726}{0.693147} = 4.00 \text{ bits/token}, \tag{1.62}$$

which confirms the identical perplexity value: $\text{PPL} = 2^{\mathcal{H}_2} = 2^{4.00} = 16.00$. Inadvertently evaluating $2^{\mathcal{L}_{\text{CE}}} = 2^{2.7726} \approx 6.83$ would severely underestimate the true branching factor by more than 57%, highlighting the necessity of matching logarithmic bases.

1.3.4 The structural ceiling and the bridge to continuous word embeddings

While advanced smoothing techniques such as Modified Kneser-Ney eliminate zero-frequency collapses and reduce empirical perplexity, statistical n-gram models remain trapped behind two insurmountable theoretical barriers:

1. **Context horizon truncation**: The $(n-1)$-th order Markov assumption discards all context residing beyond $n-1$ tokens. Increasing $n \geq 5$ yields rapidly diminishing returns because higher-order frequency tables become almost entirely empty, forcing the model to constantly back off to lower orders. Consequently, statistical n-gram models cannot maintain long-range syntactic agreement (e.g. subject-verb agreement across intervening subordinate clauses) or preserve document-level topic coherence.

2. **Discrete orthogonal representations and zero parameter sharing**: In statistical counting tables, words are represented as atomic, discrete symbolic indices. In vector space, this corresponds to treating words as standard basis vectors (one-hot vectors) in $\mathbb{R}^{|\mathcal{V}|}$:

$$\mathbf{e}_i^\top \mathbf{e}_j = 0, \quad \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}, \quad \forall i \neq j. \tag{1.63}$$

   Every word is geometrically orthogonal and equidistant to every other word. The model possesses no intrinsic notion of semantic similarity. Observing the training phrase *"The*

cat chased the mouse" provides strictly zero statistical evidence to assist the model when evaluating the semantically identical sentence *"The feline pursued the rodent"*. Because each lexical entry occupies an isolated, independent row in the transition matrix, the model cannot share statistical parameters between synonyms.

Smoothing techniques merely patch zero-frequency counts; they do not solve the fundamental lack of continuous semantic representation. Overcoming this representational ceiling requires replacing discrete, tabular counting models with continuous, parameterized mathematical functions [**?**]. Words must be projected into continuous Euclidean vector spaces $\mathbb{R}^d$ where geometric distance reflects semantic similarity. This profound conceptual leap—transitioning from discrete orthogonal indices to distributed continuous vector representations—forms the subject of Chapter **??**.